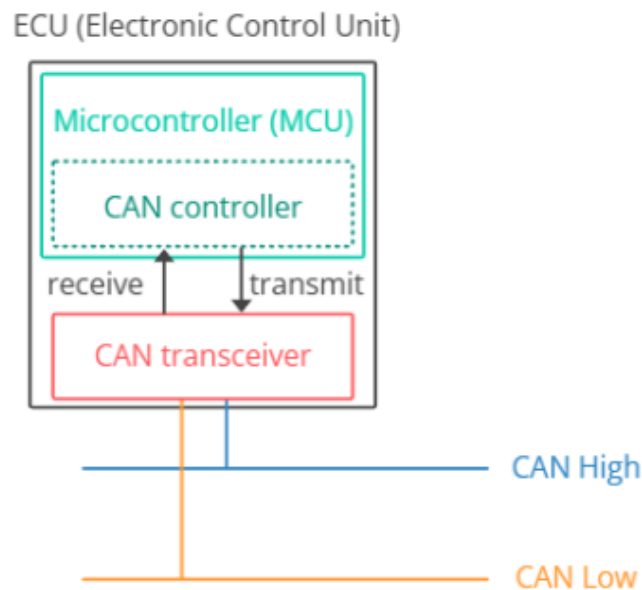


1. CAN (CONTROLLER AREA NETWORK):

1.1. Tổng quan về bxCAN STM32F103C8T6

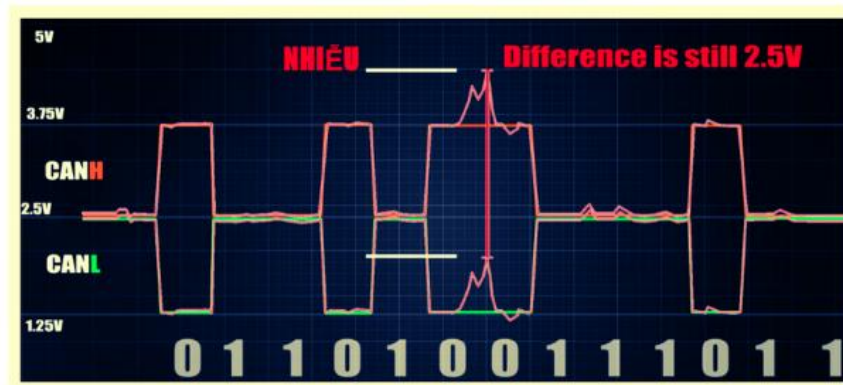
1.1.1. Cấu Trúc ECU Và Tín Hiệu Vi Sai của CANBUS:

- Hệ thống giao tiếp CAN (Controller Area Network) trên vi điều khiển STM32 được thiết kế dựa trên cấu trúc phân lớp phần cứng và tín hiệu vật lý đặc thù để đảm bảo độ tin cậy cao trong môi trường nhiễu công nghiệp.



- **Cấu tạo của một node mạng (ECU):** Một node mạng CAN (ECU - Electronic Control Unit) tiêu chuẩn bao gồm ba thành phần chính phối hợp với nhau:
 - **Microcontroller (MCU):** Khối xử lý trung tâm (ví dụ: STM32F103), nơi chạy các thuật toán điều khiển và xử lý dữ liệu ứng dụng.
 - **CAN Controller:** Nằm bên trong MCU (khối bxCAN), chịu trách nhiệm:
 - Đóng gói dữ liệu vào khung tin (Frame).
 - Kiểm soát lỗi (Error Checking).
 - Lọc mã định danh (ID Filtering).
 - Quản lý việc truyền/nhận qua các hộp thư (Mailbox) và bộ đệm (FIFO).
 - **CAN Transceiver (Bộ thu phát):** Là linh kiện trung gian kết nối giữa MCU và đường dây vật lý. Nhiệm vụ của nó là chuyển đổi tín hiệu logic từ chân Tx/Rx của vi điều khiển sang mức điện áp chênh lệch (Differential Voltage) trên hai dây CAN High và CAN Low.
- **Nguyên lý tín hiệu vi sai (Differential Signaling):** Mạng CAN sử dụng phương pháp truyền tín hiệu vi sai trên hai dây dẫn xoắn đôi (CAN High và CAN Low) để chống nhiễu hiệu quả:

- **Mức Logic 1 (Recessive - Lặn):** Cả hai dây CAN High và CAN Low đều ở mức điện áp xấp xỉ 2.5V.
- **Mức Logic 0 (Dominant - Trội):** Dây CAN High được kéo lên mức 3.5V, dây CAN Low bị kéo xuống 1.5V.
- **Khả năng chống nhiễu:** Như biểu đồ minh họa, khi có nhiễu tác động (gai điện áp), nó sẽ tác động lên cả hai dây cùng lúc. Tuy nhiên, giá trị logic được xác định bằng hiệu số (CAN_H - CAN_L). Do đó, mức chênh lệch điện áp vẫn giữ nguyên (Difference is still 2.5V), giúp dữ liệu không bị sai lệch.



1.1.2. CAN BUS và các giao thức khác:

UART	I2C/SPI	USB	CAN
KHOẢNG CÁCH Thấp (< 15 m)	KHOẢNG CÁCH Cực thấp (< 50 cm)	KHOẢNG CÁCH Thấp (< 5 m)	KHOẢNG CÁCH Cao (40m - 1Km)
TOPOLOGY Point - point	TOPOLOGY Master - Slave	TOPOLOGY Star	TOPOLOGY Multi - Master
KẾT NỐI 2 dây + GND	KẾT NỐI 2 - 4 dây + GND	KẾT NỐI 4 dây	KẾT NỐI 2 dây xoắn
CƠ CHẾ TÍN HIỆU Đơn cực (Single)	CƠ CHẾ TÍN HIỆU Đơn cực (Single)	CƠ CHẾ TÍN HIỆU Vi sai (Diff)	CƠ CHẾ TÍN HIỆU Vi sai (Diff)

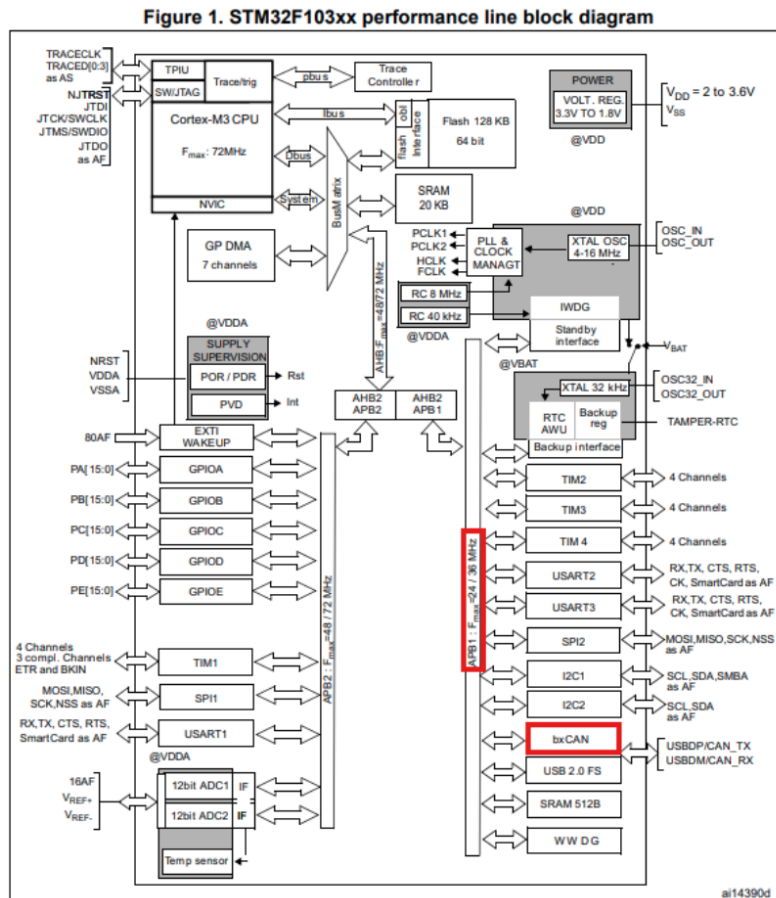
- **Về khoảng cách:** CAN hoạt động ổn định ở khoảng cách xa (40m - 1km)
- **Về khả năng chống nhiễu:** Nhờ cơ chế tín hiệu **Vi sai (Differential)** trên 2 dây xoắn, CAN có khả năng triệt tiêu nhiễu môi trường cực tốt.
- **Về cấu trúc mạng:** Kiến trúc **Multi-Master (Đa chủ)** cho phép mọi node đều có quyền truyền tin và hệ thống không bị tê liệt.
- **Về kết nối:** Chỉ cần **2 dây** để kết nối nhiều thiết bị, giúp tối ưu hóa chi phí và giảm trọng lượng dây dẫn trong hệ thống.

1.1.3. Giới thiệu về bxCan:

- Trên dòng vi điều khiển STM32, ngoại vi chịu trách nhiệm giao tiếp CAN được gọi là bxCAN (Basic Extended Controller Area Network).

- Các đặc điểm kỹ thuật chính của bxCAN bao gồm:
 - **Chuẩn giao tiếp hỗ trợ:** bxCAN hỗ trợ đầy đủ cả hai chuẩn **CAN 2.0A** (sử dụng mã định danh Standard ID 11-bit) và **CAN 2.0B** (sử dụng mã định danh Extended ID 29-bit) ở chế độ Active. Điều này cho phép vi điều khiển tương thích với hầu hết các cảm biến và ECU hiện có trên thị trường.
 - **Tốc độ truyền:** Hỗ trợ tốc độ truyền dữ liệu (Bit rate) tối đa lên đến **1 Mbit/s**, đáp ứng tiêu chuẩn High-Speed CAN (ISO 11898).
 - **Mô hình mạng (Topology):** Hoạt động theo cơ chế **Multi-master** (Đa chủ), cho phép thiết bị tự chủ động gửi dữ liệu khi đường truyền rảnh mà không cần chờ lệnh từ một thiết bị chủ trung tâm.

1.1.4. bxCan trên STM32F103C8T6:



- **Tài nguyên phần cứng:**
 - **Số lượng bộ điều khiển:** Chip STM32F103C8T6 chỉ được tích hợp duy nhất 1 bộ CAN Controller (gọi là CAN1).
 - **Bộ lọc phần cứng (Filter Banks):** Cung cấp **14 bộ lọc (Filter Banks)**, được đánh số từ Bank 0 đến Bank 13. Các bộ lọc này giúp CPU không bị quá tải

bằng cách chỉ cho phép các tin nhắn có ID mong muốn đi vào bộ nhớ nhận (FIFO).

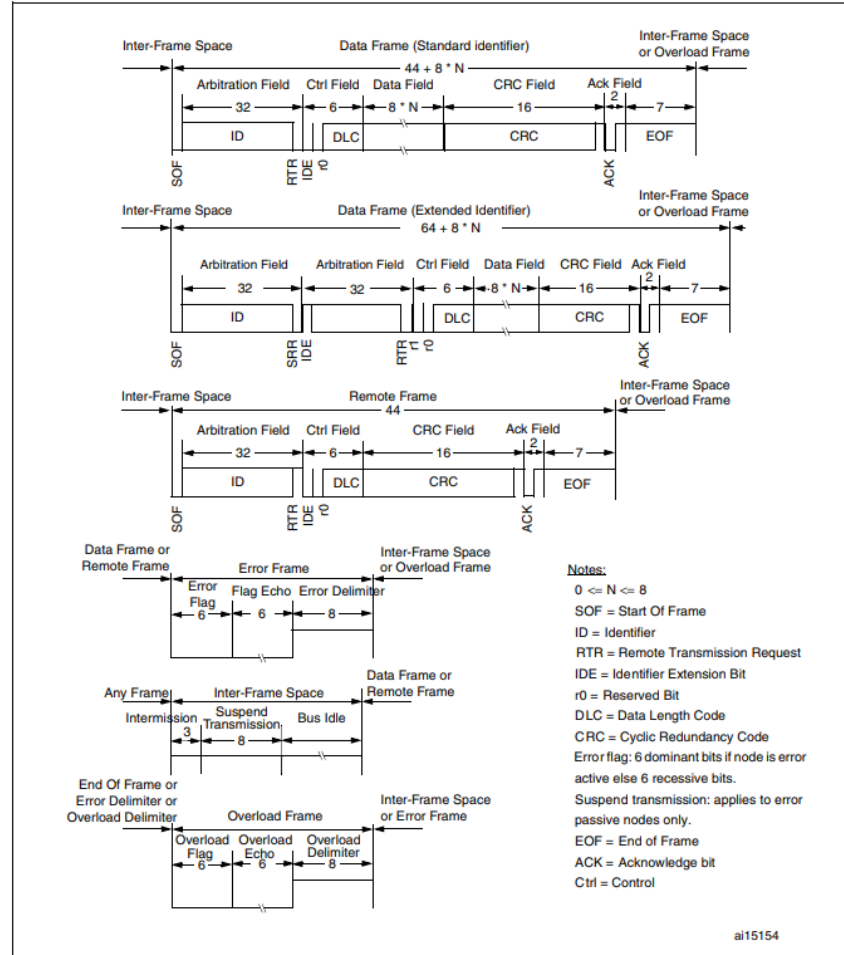
- **Bus kết nối:** Ngoại vi bxCAN được kết nối vào bus **APB1**. Điều này đồng nghĩa với việc tần số xung nhịp cấp cho khối CAN bị giới hạn ở mức tối đa là **36 MHz** (do APB1 có tốc độ tối đa 36 MHz trên dòng F103 chạy 72 MHz). Đây là thông số quan trọng để tính toán Baud rate.
- **Xung đột tài nguyên với USB:** Một đặc điểm thiết kế phần cứng đặc thù của dòng STM32F103 là sự chia sẻ tài nguyên giữa CAN và USB:
 - **Chia sẻ bộ nhớ đệm:** CAN và USB dùng chung một vùng nhớ **SRAM 512-byte** chuyên dụng.
⇒ **Hệ quả: Không thể sử dụng CAN và USB cùng một lúc.**
- **Cấu hình chân CAN và Remap:** Do chân mặc định của CAN trùng với chân của USB, việc cấu hình chân (GPIO Remapping) là bước bắt buộc nếu muốn tín hiệu CAN ổn định và không bị ảnh hưởng bởi chân USB.

Table 30. CAN1 alternate function remapping

Alternate function ⁽¹⁾	CAN_REMAP[1:0] = "00"	CAN_REMAP[1:0] = "10" ⁽²⁾	CAN_REMAP[1:0] = "11" ⁽³⁾
CAN1_RX or CAN_RX	PA11	PB8	PD0
CAN1_TX or CAN_RX	PA12	PB9	PD1

1.2. Cấu trúc gói tin CanBus

Figure 205. CAN frames



- **Trường khởi tạo và Định danh (Arbitration Field):**
 - **SOF (Start Of Frame) - 1 bit:**
 - **Giá trị:** Luôn là mức Dominant (Logic 0).
 - **Chức năng:** Đánh dấu sự bắt đầu của một khung tin. Khi đường truyền đang rảnh (Idle - mức 1), việc xuất hiện một bit 0 (SOF) sẽ kích hoạt cơ chế đồng bộ cứng, báo hiệu cho tất cả các Node trên mạng biết rằng có một dữ liệu đang được gửi tới để chúng bắt đầu lắng nghe.
 - **Identifier (ID) - 11 bit (Standard) hoặc 29 bit (Extended):**
 - **Chức năng:** Định danh nội dung của gói tin. Khác với các giao thức khác dùng địa chỉ Node, CAN dùng địa chỉ nội dung.
 - **Cơ chế Trọng tài (Arbitration):** Đây là cơ chế cốt lõi giúp CAN hoạt động ở chế độ Đa chủ (Multi-master).
 - Khi gửi từng bit của ID, bộ điều khiển CAN đồng thời "nghe" lại trạng thái trên đường dây.

- Nếu Node A phát mức **Recessive (1)** nhưng lại thấy đường dây bị kéo xuống **Dominant (0)** (do Node B phát), Node A hiểu rằng có một Node khác có độ ưu tiên cao hơn đang truyền.

⇒ ID có giá trị càng nhỏ thì độ ưu tiên càng lớn (do chứa nhiều bit 0 ở đầu).

- **RTR (Remote Transmission Request) - 1 bit:**

- **Giá trị 0 (Dominant):** Data Frame - Khung chứa dữ liệu thực.
- **Giá trị 1 (Recessive):** Remote Frame - Khung yêu cầu dữ liệu (Dùng để Master yêu cầu Slave gửi lại thông số mà không cần gửi data đi).

- **Trường điều khiển (Control Field):**

- **IDE (Identifier Extension) - 1 bit:**

- **Giá trị:**
 - **IDE = 0:** Standard Frame (Sử dụng ID 11-bit).
 - **IDE = 1:** Extended Frame (Sử dụng ID 29-bit).
- **Chức năng:** Phân biệt định dạng khung tin.

- **Reserved Bit (r0) - 1 bit:**

- **Giá trị:** Luôn là mức Dominant (0).
- **Chức năng:** Bit dự trữ cho các nâng cấp giao thức trong tương lai.

- **DLC (Data Length Code) - 4 bit:**

- **Giá trị:** 0 => 8.
- **Chức năng:** Thông báo cho bên nhận biết chính xác số lượng byte dữ liệu (Payload) sẽ theo sau.

- **Trường dữ liệu (Data Field):**

- **Data Field:**

- **Giá trị:** Từ 0 đến 64 bit (tương ứng 0 đến 8 byte).
- **Chức năng:** Chứa thông tin thực tế cần truyền tải (ví dụ: giá trị cảm biến, lệnh điều khiển). Byte đầu tiên (MSB) được truyền trước.

- **Trường kiểm soát lỗi và Kết thúc:**

- **CRC Field (Cyclic Redundancy Check) - 15 bit + 1 bit Delimiter:**

- **Chức năng:** Kiểm tra tính toàn vẹn. Người gửi tính toán giá trị CRC dựa trên toàn bộ dữ liệu phía trước và gửi kèm.
- **Cơ chế:** Bên nhận sẽ tự tính toán lại CRC theo cùng công thức.
 - Nếu kết quả trùng khớp: Gói tin sạch, không lỗi.
 - Nếu kết quả lệch: Gói tin bị nhiễu, bên nhận sẽ hủy gói tin và phát cờ báo lỗi (Error Frame) để yêu cầu gửi lại.

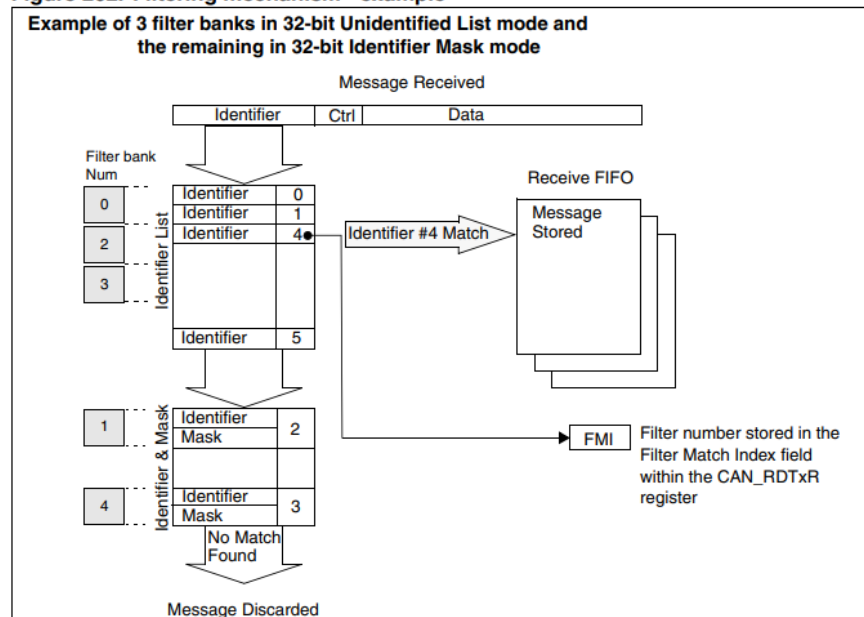
- **ACK Field (Acknowledge) - 1 bit + 1 bit Delimiter:**
 - **Giá trị:** 0 => 8.
 - **Cơ chế bắt tay (Handshaking):**
 - **Người gửi:** Luôn phát mức Recessive (1) tại khe ACK Slot.
 - **Người nhận:** Nếu bất kỳ Node nào trên mạng nhận được gói tin và kiểm tra CRC thấy đúng, nó sẽ kéo đường dây xuống mức Dominant (0) ngay tại khe này.
 - **Kết quả:**
 - Người gửi thấy đường dây về 0 => "Gửi thành công, đã có ít nhất một người nhận".
 - Người gửi thấy đường dây vẫn là 1 => "Không ai nhận cả" hoặc "Tất cả đều thấy lỗi" => Tự động gửi lại.
- **EOF (End Of Frame) - 7 bit:**
 - **Giá trị:** Luôn là 7 bit mức **Recessive (1111111)**.
 - **Chức năng:** Đánh dấu chấm hết cho một khung tin, đưa đường truyền về trạng thái nghỉ (Idle) hoặc sẵn sàng cho khung tin tiếp theo.

1.3. Identifier Filtering

1.3.1. Vấn đề của Broadcast & Giải pháp:

- **Thách thức:** Trong mạng CAN, không có địa chỉ đích cụ thể cho từng node. Khi một node gửi dữ liệu, tất cả các node khác trên mạng đều nhận được tín hiệu đó.
 - **Vấn đề:** Nếu một hệ thống có hàng trăm tin nhắn được gửi liên tục (ví dụ: tốc độ động cơ, nhiệt độ, vị trí chân ga...), CPU của vi điều khiển sẽ bị ngắt (interrupt) liên tục để kiểm tra từng gói tin xem có liên quan đến mình hay không.
 - **Hệ quả:** Tài nguyên CPU bị lãng phí cho việc xử lý các tin nhắn rác (không cần thiết), làm giảm hiệu năng của các tác vụ chính khác.
- **Giải pháp:** Để khắc phục vấn đề trên, STM32 (và các bộ điều khiển CAN tiêu chuẩn) tích hợp sẵn các **Filter Bank (Ngân hàng lọc)** nằm ngay tại phần cứng, hoạt động độc lập với CPU.

Figure 202. Filtering mechanism - example



- **Quy trình xử lý:**

- **Sàng lọc:** Khi một gói tin đi vào từ đường bus, bộ điều khiển CAN sẽ so sánh ID của gói tin đó với các ID đã được cài đặt trong các Filter Bank.
- **Quyết định:**
 - **Match (Khớp):** Nếu ID gói tin trùng khớp với bộ lọc, gói tin được phép đi qua và lưu vào bộ đệm nhận (Receive FIFO). Lúc này, CPU mới nhận được tín hiệu ngắt để đọc dữ liệu.
 - **No Match (Không khớp):** Nếu ID không thỏa mãn bất kỳ bộ lọc nào, gói tin sẽ bị vứt bỏ ngay lập tức (Message Discarded) tại phần cứng. CPU hoàn toàn không biết đến sự tồn tại của gói tin này, giúp tiết kiệm tối đa tài nguyên xử lý.

1.3.2. Các chế độ lọc:

- Để đáp ứng linh hoạt các nhu cầu giao tiếp khác nhau, bộ điều khiển bxCAN trên STM32 cung cấp hai chế độ cấu hình cho mỗi Filter Bank: Identifier List Mode (Chế độ danh sách) và Identifier Mask Mode (Chế độ mặt nạ). Việc lựa chọn chế độ phù hợp giúp tối ưu hóa tài nguyên phần cứng vốn hạn chế (chỉ có 14 Filter Banks trên STM32F103).

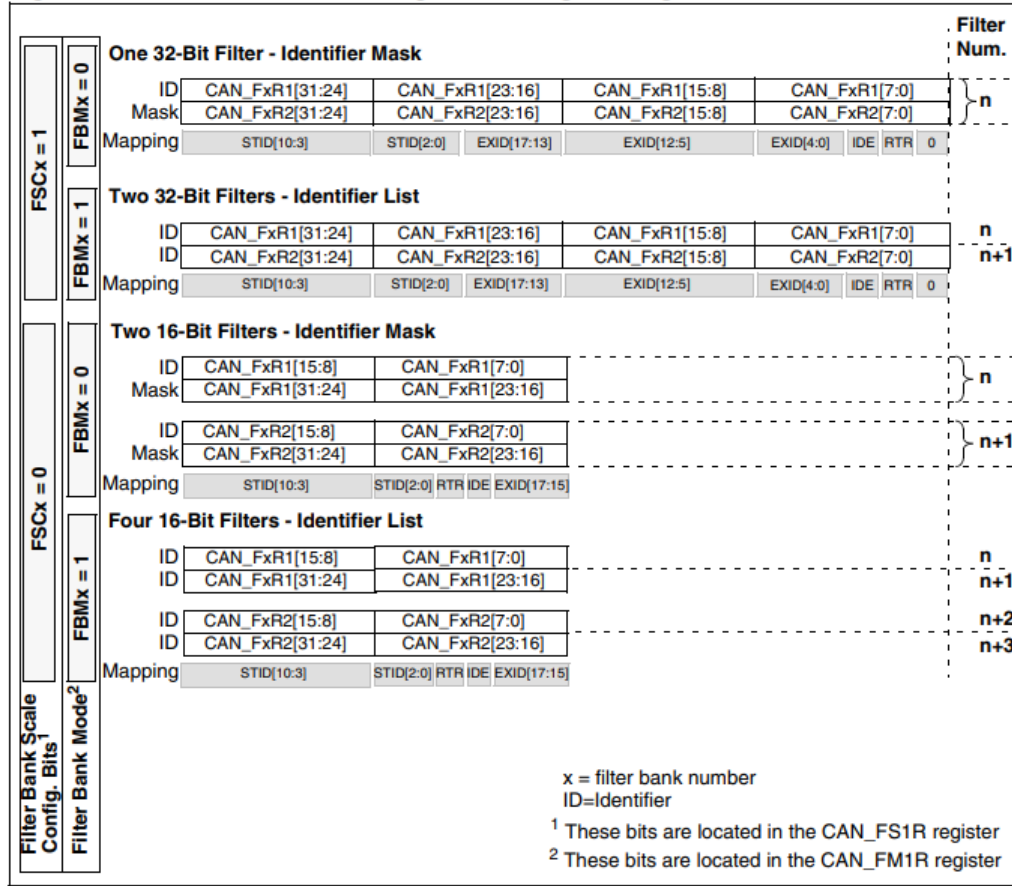
- **Chế độ Danh sách (Identifier List Mode):** Đây là chế độ lọc đơn giản và trực quan nhất.
 - **Nguyên tắc hoạt động:** Bộ lọc hoạt động như một "danh sách trắng" (Whitelist). Phần cứng sẽ so sánh chính xác tuyệt đối (100%) từng bit của ID nhận được với các ID đã được nạp trong thanh ghi bộ lọc.

- **Cấu hình:** Trong chế độ này, mỗi thanh ghi 32-bit (hoặc 16-bit) sẽ chứa một ID cụ thể. Ví dụ: Nếu cấu hình Bank 0 ở chế độ List 16-bit, ta có thể lưu được 4 ID riêng biệt.
- **Ưu điểm:** Độ chính xác tuyệt đối. Node mạng sẽ không bao giờ phải xử lý các tin nhắn không mong muốn. Cấu hình đơn giản, dễ hiểu.
- **Nhược điểm:** Tiêu tốn tài nguyên bộ lọc. Nếu ứng dụng yêu cầu nhận 50 loại tin nhắn khác nhau, ta sẽ cần rất nhiều Filter Bank, điều mà phần cứng thường không đáp ứng đủ.
- **Chế độ Mặt nạ (Identifier Mask Mode):** Đây là chế độ lọc mạnh mẽ và linh hoạt nhất của CAN, cho phép một bộ lọc duy nhất có thể bắt được cả một nhóm các ID có đặc điểm chung.
 - **Nguyên tắc hoạt động:** Thay vì so sánh toàn bộ ID, chế độ này sử dụng hai thanh ghi phối hợp:
 - **Identifier:** Chứa mẫu bit cần so sánh.
 - **Mask Register:** Quy định bit nào là quan trọng.
 - **Quy luật Logic của Mask:**
 - **Mask Bit = 1 (Must Match):** Bắt buộc bit tương ứng của ID nhận được phải **giống hệt** bit trong thanh ghi Identifier. Nếu sai, gói tin bị loại.
 - **Mask Bit = 0 (Don't Care):** Không quan tâm bit của ID nhận được là 0 hay 1 (chấp nhận tất cả).
 - **Ưu điểm:** Cực kỳ linh hoạt và tiết kiệm tài nguyên. Chỉ cần 1 Filter Bank có thể lọc được hàng chục, thậm chí hàng trăm ID khác nhau nếu chúng được quy hoạch khéo léo (có chung các bit đầu).
 - **Nhược điểm:** Yêu cầu tư duy logic chặt chẽ khi tính toán giá trị Mask. Nếu cấu hình sai, hệ thống có thể nhận nhầm tin nhắn rác hoặc bỏ lỡ tin nhắn quan trọng.
- **Ví dụ tính toán bộ lọc Mask Mode:** Giả sử hệ thống cần nhận một dải gồm 16 tin nhắn có ID từ 0x120 đến 0x12F.
 - **Phân tích nhị phân (với ID 11-bit):**
 - ID bắt đầu (0x120): 001 0010 0000
 - ID kết thúc (0x12F): 001 0010 1111
 - **Nhận xét:**
 - 7 bit đầu (001 0010): Giống hệt nhau ở tất cả các ID trong dải. Đây là phần cần so sánh chính xác => Mask bit phải là 1.
 - 4 bit cuối (0000 đến 1111): Thay đổi liên tục. Đây là phần không quan tâm => Mask bit phải là 0.

- **Kết quả cấu hình:**
 - ID (0x120): 001 0010 0000
 - Mask (0x7F0): 111 1111 0000

1.3.3. Filter bank scale & Mode:

Figure 200. Filter bank scale configuration - register organization



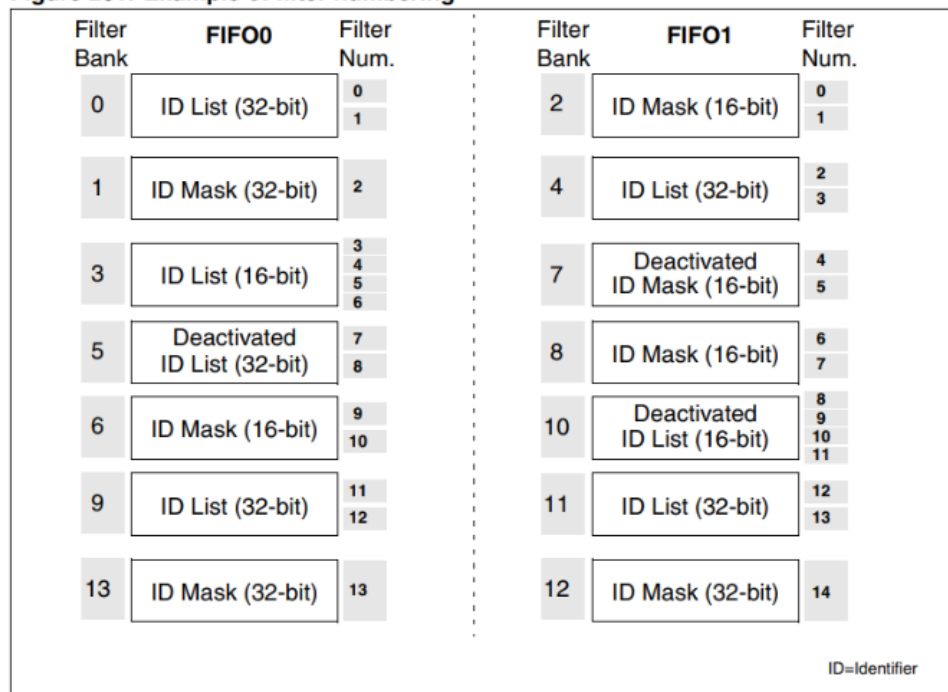
- Để tối ưu hóa tài nguyên phần cứng, STM32 cho phép mỗi Filter Bank có thể được cấu hình linh hoạt về độ rộng (Scale) là 32-bit hoặc 16-bit, kết hợp với chế độ Lọc (Mask/List). Điều này tạo ra 4 trường hợp cấu hình chính:
 - **Chế độ 32-bit:** Chế độ này sử dụng toàn bộ thanh ghi 32-bit, thường được dùng khi làm việc với Extended ID (29-bit) vì ID này quá dài để chứa trong thanh ghi 16-bit.
 - **32-bit Mask Mode:**
 - **Cấu hình:** Bit FSCx = 1 (32-bit) và FBMx = 0 (Mask Mode).
 - **Cơ chế:** Một Filter Bank sẽ tạo thành **1 bộ lọc duy nhất**.

- + Thanh ghi FxR1: Chứa Identifier.
 - + Thanh ghi FxR2: Chứa Mask.
 - **Ứng dụng:** Dùng để lọc một **nhóm** các Extended ID.
- **32-bit List Mode:**
 - **Cấu hình:** Bit FSCx = 1 (32-bit) và FBMx = 1 (List Mode).
 - **Cơ chế:** Một Filter Bank sẽ chứa được 2 ID cụ thể.
 - + Thanh ghi FxR1: Chứa ID chính xác thứ 1.
 - + Thanh ghi FxR2: Chứa ID chính xác thứ 2.
 - **Ứng dụng:** Dùng để bắt chính xác các Extended ID cụ thể.
- **Chế độ 16-bit:** Chế độ này chia đôi thanh ghi 32-bit thành hai phần 16-bit độc lập. Đây là chế độ tiết kiệm tài nguyên nhất khi làm việc với **Standard ID (11-bit)**.
 - **16-bit Mask Mode:**
 - **Cấu hình:** Bit FSCx = 0 (16-bit) và FBMx = 0 (Mask Mode).
 - **Cơ chế:** Bit FSCx = 0 (16-bit) và FBMx = 0 (Mask Mode).
 - + Thanh ghi FxR1: Chia thành [Mask 1] và [ID 1].
 - + Thanh ghi FxR2: Chia thành [Mask 2] và [ID 2].
 - **Ứng dụng:** Dùng để lọc 2 nhóm Standard ID khác nhau trong cùng 1 bank.
 - **16-bit List Mode:**
 - **Cấu hình:** Bit FSCx = 0 (16-bit) và FBMx = 1 (List Mode).
 - **Cơ chế:** Một Filter Bank chứa được **4 ID cụ thể**.
 - + Thanh ghi FxR1: Chứa [ID 2] và [ID 1].
 - + Thanh ghi FxR2: Chứa [ID 4] và [ID 3].
 - **Ứng dụng:** Dùng để bắt chính xác tối đa 4 Standard ID khác nhau. Đây là chế độ hiệu quả nhất về mặt số lượng ID lọc được.
- **Bản đồ ánh xạ thanh ghi:** Việc nạp giá trị ID (ví dụ 0x123) vào thanh ghi bộ lọc không phải là ghi trực tiếp giá trị đó, mà cần phải tuân theo cấu trúc sắp xếp bit (Mapping) của phần cứng bxCAN.
 - **Ánh xạ Standard ID (Trong chế độ 16-bit):** Ví dụ: Cần nạp ID 0x123 (001 0010 0011) vào bộ lọc để chỉ nhận Data Frame chuẩn.
 - **Cấu trúc thanh ghi 16-bit:**[STID 11bit] [RTR] [IDE] [EXID 3bit]
 - **Phép toán dịch bit:**
 - + **Dịch trái 5 bit (ID << 5):** Đưa 11 bit ID lên vị trí cao nhất (bit 15 đến bit 5).

- + **Cấu hình bit IDE (Bit 3):** Set bằng 0 để báo hiệu đây là Standard Frame.
- + **Cấu hình bit RTR (Bit 4):** Set bằng 0 để chỉ nhận Data Frame (nếu muốn nhận Remote Frame thì set bằng 1).
- **Kết quả Binary:** 0010 0100 0110 0000.
- **Ánh xạ Extended ID (Trong chế độ 32-bit):** Ví dụ: Cần nạp ID 0x10001 (0001 ... 0001) vào bộ lọc để nhận Extended Frame.
 - **Cấu trúc thanh ghi 32-bit:** [EXID 29-bit] [IDE] [RTR] [0]
 - **Phép toán dịch bit:**
 - + **Dịch trái 3 bit (ID < 3):** Đưa 29 bit ID lên vị trí cao nhất (bit 31 đến bit 3).
 - + **Cấu hình bit IDE (Bit 2):** Set bằng 1 (hoặc | 0x04) để báo hiệu đây là Extended Frame.
 - + **Cấu hình bit RTR (Bit 1):** Set bằng 0 cho Data Frame.
 - **Kết quả Binary:** 0001 ... 0000 0000 1100.

1.3.4. Phân luồng FIFO & Filter Match Index:

Figure 201. Example of filter numbering



- **Phân luồng dữ liệu:** Bộ điều khiển CAN trên STM32 cung cấp 2 bộ đệm nhận độc lập là **FIFO 0** và **FIFO 1**. Việc tận dụng cả hai FIFO này giúp lập trình viên tối ưu hóa độ trễ xử lý và phân loại mức độ ưu tiên của tin nhắn ngay từ phần cứng.

[illegible]

- **Bit FSCx:** Cấu hình độ rộng cho Filter Bank x.
 - FSCx = 0: Chế độ **16-bit** Scale.
 - FSCx = 1: Chế độ **32-bit** Scale.

- **Can filter mode register [CAN_FM1R]:**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FBM27	FBM26	FBM25	FBM24	FBM23	FBM22	FBM21	FBM20	FBM19	FBM18	FBM17	FBM16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FBM15	FBM14	FBM13	FBM12	FBM11	FBM10	FBM9	FBM8	FBM7	FBM6	FBM5	FBM4	FBM3	FBM2	FBM1	FBM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- **Bit FBMx:** Cấu hình chế độ lọc cho Filter Bank x.
 - FBMx = 0: Chế độ Identifier Mask mode.
 - FBMx = 1: Chế độ Identifier List mode.

- **Can filter FIFO assignment register [CAN_FFA1R]:**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FFA27	FFA26	FFA25	FFA24	FFA23	FFA22	FFA21	FFA20	FFA19	FFA18	FFA17	FFA16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FFA15	FFA14	FFA13	FFA12	FFA11	FFA10	FFA9	FFA8	FFA7	FFA6	FFA5	FFA4	FFA3	FFA2	FFA1	FFA0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- **Bit FFAx:** Gán Filter Bank x cho FIFO nào.
 - FFAx = 0: Gán bộ lọc này cho FIFO 0.
 - FFAx = 1: Gán bộ lọc này cho FIFO 1.

- **Filter bank i register x [CAN_FiRx]:**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FB31	FB30	FB29	FB28	FB27	FB26	FB25	FB24	FB23	FB22	FB21	FB20	FB19	FB18	FB17	FB16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FB15	FB14	FB13	FB12	FB11	FB10	FB9	FB8	FB7	FB6	FB5	FB4	FB3	FB2	FB1	FB0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- **CAN_FR1/2 :** Lưu id lọc và bit **FB** là từng bit trong các thanh ghi FR.

MODE	FR1 (32-bit)	FR2 (32-bit)
32-bit MASK	ID	Mask
32-bit LIST	ID 1	ID 2
16-bit MASK	[H]: Mask 1 [L]: ID 1	[H]: Mask 2 [L]: ID 2
16-bit LIST	[H]: ID 2 [L]: ID 1	[H]: ID 4 [L]: ID 3

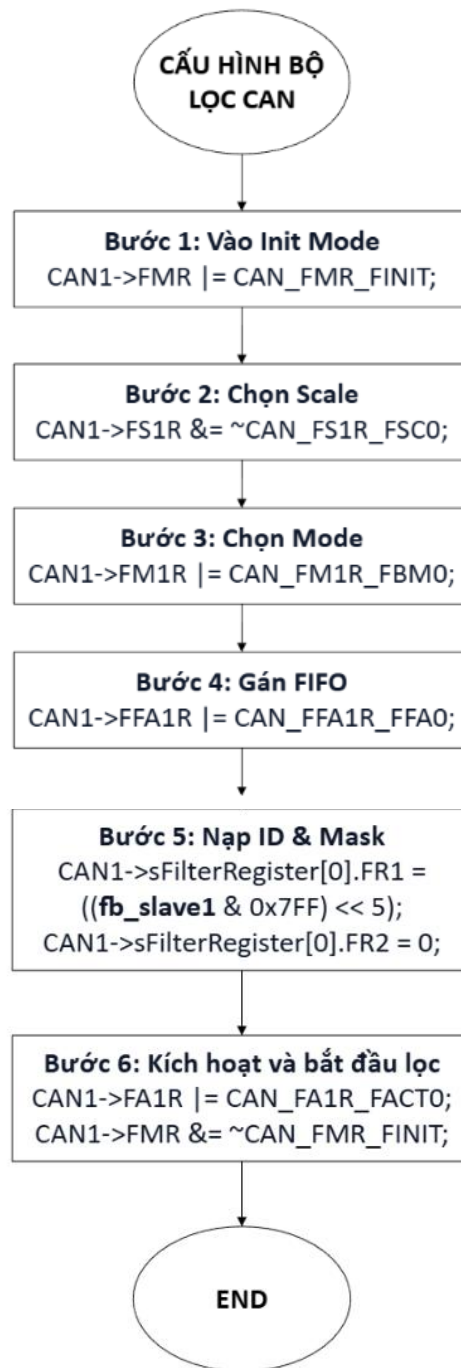
- **Can filter activation register [CAN_FA1R]:**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FACT27	FACT26	FACT25	FACT24	FACT23	FACT22	FACT21	FACT20	FACT19	FACT18	FACT17	FACT16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FACT15	FACT14	FACT13	FACT12	FACT11	FACT10	FACT9	FACT8	FACT7	FACT6	FACT5	FACT4	FACT3	FACT2	FACT1	FACT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

○ **Bit FACTx: Kích hoạt Filter Bank x.**

- FACTx = 0: Tắt bộ lọc Bank x.
- FACTx = 1: Bật bộ lọc Bank x.

- Các bước cấu hình bộ lọc:



- Chương trình cấu hình bộ lọc của dự án:

```
/**
 * @brief Cấu hình bộ lọc CAN MASTER
 * @note Bank 0: Loc ID Slave 1 (List mode)
 * @note Bank 1: Loc ID Slave 2 (List mode)
 */
void can_master_setup_filters(can_feedback_t fb_slave1, can_feedback_t fb_slave2)
{
    CAN1->FMR |= CAN_FMR_FINIT; // Vào chế độ cài đặt filter
    CAN1->FA1R = 0; // Tắt tất cả filter để cấu hình
    // --- Bank 0: Loc ID Slave 1 ---
    CAN1->FS1R &= ~CAN_FS1R_FSC0; // Chọn chế độ 16-bit
    CAN1->FM1R |= CAN_FM1R_FBM0; // Chọn chế độ List (danh sách cụ thể)
    CAN1->FFA1R |= CAN_FFA1R_FFA0; // Gán vào FIFO 1
    CAN1->sFilterRegister[0].FR1 = ((fb_slave1 & 0x7FF) << 5); // Nạp ID
    CAN1->sFilterRegister[0].FR2 = 0;
    CAN1->FA1R |= CAN_FA1R_FACT0; // Kích hoạt filter 0

    // --- Bank 1: Loc ID Slave 2 ---
    CAN1->FS1R &= ~CAN_FS1R_FSC1; // Chọn chế độ 16-bit
    CAN1->FM1R |= CAN_FM1R_FBM1; // Chọn chế độ List
    CAN1->FFA1R |= CAN_FFA1R_FFA1; // Gán vào FIFO 1
    CAN1->sFilterRegister[1].FR1 = ((fb_slave2 & 0x7FF) << 5); // Nạp ID
    CAN1->sFilterRegister[1].FR2 = 0;
    CAN1->FA1R |= CAN_FA1R_FACT1; // Kích hoạt filter 1

    CAN1->FMR &= ~CAN_FMR_FINIT; // Thoát chế độ cài đặt, bắt đầu lọc
}
/**
 * @brief Cấu hình bộ lọc CAN SLAVE (2 Banks)
 * @note Bank 0: Mask mode (Loc theo đại ID)
 * @note Bank 1: List mode (Loc lệnh START và STOP)
 * @note Bank 2: List mode (Loc lệnh SETPOINT riêng)
 */
void can_slave_setup_filters(can_command_t cmd_start, can_command_t cmd_stop, can_command_t
cmd_setpoint, can_command_t cmd_all)
{
    CAN1->FMR |= CAN_FMR_FINIT; // Vào chế độ cài đặt filter
    CAN1->FA1R = 0; // Tắt tất cả filter
    // --- Bank 0: Mask mode (Loc đại ID chung) ---
```

```

CAN1->FS1R |= CAN_FS1R_FSC0; // Chon che do 32-bit
CAN1->FM1R &= ~CAN_FM1R_FBM0; // Chon che do Mask (mat na)
CAN1->FFA1R |= CAN_FFA1R_FFA0; // Gan vao FIFO 1
// Nap ID mau va Mask (loc theo bit cao)
CAN1->sFilterRegister[0].FR1 = (cmd_all << 21); // ID mau
CAN1->sFilterRegister[0].FR2 = (0x7F0 << 21); // Mask
CAN1->FA1R |= CAN_FA1R_FACT0; // Kich hoat filter 0
// --- Bank 1: List mode (Loc START va STOP) ---
CAN1->FS1R &= ~CAN_FS1R_FSC1; // Chon che do 16-bit
CAN1->FM1R |= CAN_FM1R_FBM1; // Chon che do List
CAN1->FFA1R |= CAN_FFA1R_FFA1; // Gan vao FIFO 1
// Nap 2 ID vao cung mot thanh ghi 32-bit (chia lam 2 phan 16-bit)
CAN1->sFilterRegister[1].FR1 =
((cmd_start & 0x7FF) << 21) | ((cmd_stop & 0x7FF) << 5);
CAN1->sFilterRegister[1].FR2 = 0;
CAN1->FA1R |= CAN_FA1R_FACT1; // Kich hoat filter 1
// --- Bank 2: List mode (Loc SETPOINT rieng) ---
CAN1->FS1R &= ~CAN_FS1R_FSC2; // Chon che do 16-bit
CAN1->FM1R |= CAN_FM1R_FBM2; // Chon che do List
CAN1->FFA1R |= CAN_FFA1R_FFA2; // Gan vao FIFO 1
// Nap ID Setpoint vao thanh ghi
CAN1->sFilterRegister[2].FR1 = ((cmd_setpoint & 0x7FF) << 21);
CAN1->sFilterRegister[2].FR2 = 0;
CAN1->FA1R |= CAN_FA1R_FACT2; // Kich hoat filter 2
CAN1->FMR &= ~CAN_FMR_FINIT; // Thoat che do cai dat
}

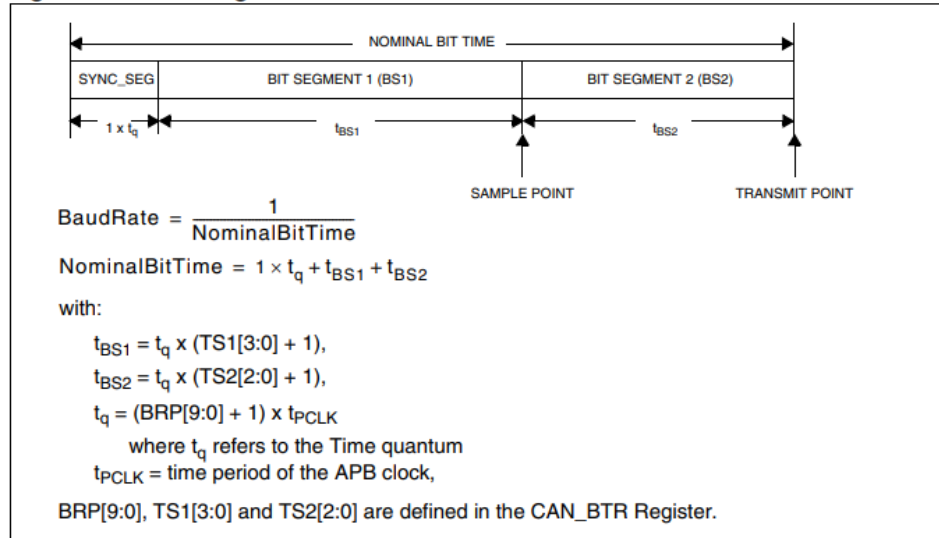
```

1.4. Bit Timing

1.4.1. Định nghĩa và thanh ghi Bit Timing CAN_BTR:

- Trong giao thức CAN, do không có dây tín hiệu xung nhịp (Clock) đi kèm để đồng bộ hóa như I2C hay SPI, các Node trên mạng phải tự thỏa thuận với nhau về tốc độ truyền và điểm lấy mẫu dữ liệu.
- Bộ điều khiển bxCAN chia một chu kỳ bit (Nominal Bit Time) thành các đoạn nhỏ dựa trên một đơn vị thời gian cơ sở gọi là **Time Quantum**.

Figure 204. Bit timing



- **Đơn vị thời gian chuẩn hóa:** Tất cả các thông số thời gian trong CAN đều là bội số của t_q . Giá trị này được tạo ra từ xung nhịp của bus hệ thống (APB1 Clock) thông qua bộ chia tần số (Prescaler).
 - **Công thức:**

$$t_q = (\text{BRP} [9: 0] + 1) \times t_{PCLK}$$
 - t_{PCLK} : Chu kỳ xung nhịp của bus APB1.
 - **BRP (Baud Rate Prescaler):** Giá trị bộ chia tần số cài đặt trong thanh ghi CAN_BTR (từ 0 đến 1023).
- **Cấu trúc của một Bit (Nominal Bit Time):** Một bit dữ liệu trên đường truyền CAN không phải là một xung đơn thuần mà được chia thành 3 đoạn thời gian chính để phục vụ việc đồng bộ hóa:
 - **SYNC_SEG (Synchronization Segment):**
 - **Độ dài:** Cố định là $1 \times t_q$
 - **Chức năng:** Đoạn này dùng để đồng bộ hóa. Các cạnh lên/xuống của tín hiệu trên bus được mong đợi sẽ xảy ra trong khoảng này.
 - **BS1 (Bit Segment 1):**
 - **Độ dài:** Được cấu hình bởi tham số TS1.

$$t_{BS1} = t_q \times (TS1[3:0] + 1)$$

- **Chức năng:** Đây là khoảng thời gian trước điểm lấy mẫu (Sample Point). Phần cứng có thể tự động kéo dài đoạn này (dẫn ra) để bù đắp nếu tín hiệu đến trễ hơn dự kiến. Nó bao gồm cả Prop_Seg (bù trễ đường truyền) và Phase_Seg1 (bù sai lệch pha)

○ **BS2 (Bit Segment 2):**

- **Độ dài:** Được cấu hình bởi tham số TS2.

$$t_{BS2} = t_q \times (TS2[2:0] + 1)$$

- **Chức năng:** Đây là khoảng thời gian **sau điểm lấy mẫu**. Phần cứng có thể tự động rút ngắn đoạn này (co lại) nếu tín hiệu đến sớm hơn dự kiến.

⇒ **Điểm lấy mẫu (Sample Point):** Nằm ngay giữa BS1 và BS2. Tại thời điểm này, vi điều khiển sẽ đọc trạng thái điện áp trên bus để quyết định bit đó là 0 hay 1.

- **Công thức tính Baud Rate:** Tốc độ truyền (Baud Rate) được xác định bởi tổng thời gian của một bit:

○ **Thời gian một bit (NominalBitTime):**

$$\begin{aligned} NominalBitTime &= 1 \times t_q + t_{BS1} + t_{BS2} \\ &= t_q \times (1 + (TS1 + 1) + (TS2 + 1)) \end{aligned}$$

○ **Tốc độ truyền (BaudRate):**

$$BaudRate = \frac{1}{NominalBitTime}$$

- **Cơ chế tái đồng bộ (SJW - Resynchronization Jump Width):** Trong môi trường thực tế, xung nhịp giữa các thành phần của các Node có thể bị sai lệch nhỏ. SJW là cơ chế "an toàn" để bù đắp sự sai lệch này.

- **Định nghĩa:** SJW quy định số lượng t_q tối đa mà phần cứng được phép thêm vào BS1 hoặc bớt đi ở BS2 trong mỗi bit để lấy lại đồng bộ.

- **Lưu ý:** Giá trị SJW không được lớn hơn độ dài của BS1 hoặc BS2.

○ **Công thức:**

$$t_{SJW} = t_q \times (SJW[1:0] + 1)$$

1.4.2. Chương trình ví dụ cấu hình Bit Timing:

/**

```

* @brief Cau hình bit timing @ 72MHz APB1 = 36MHz cho 500kbps
*/
static void can_configure_bit_timing(void)
{
    // Reset register BTR
    CAN1->BTR = 0;

    // BRP = 4 (3+1) - PRESCALER
    // t_q = 4 * 1 / 36 x 10^6
    CAN1->BTR |= (3 << 0);

    // TS1 = 15 (14+1) - TIME SEGMENT 1
    CAN1->BTR |= (14 << 16);

    // TS2 = 2 (1+1) - TIME SEGMENT 2
    CAN1->BTR |= (1 << 20);

    // SJW = 1 (0+1) - SYNC JUMP WIDTH
    CAN1->BTR |= (0 << 24);

    // NominalBitTime = t_q x (1 + ts1 + ts2) = 18 x t_q
    // => Baud rate = 1 / 18 x t_q = 500 000 bps
}

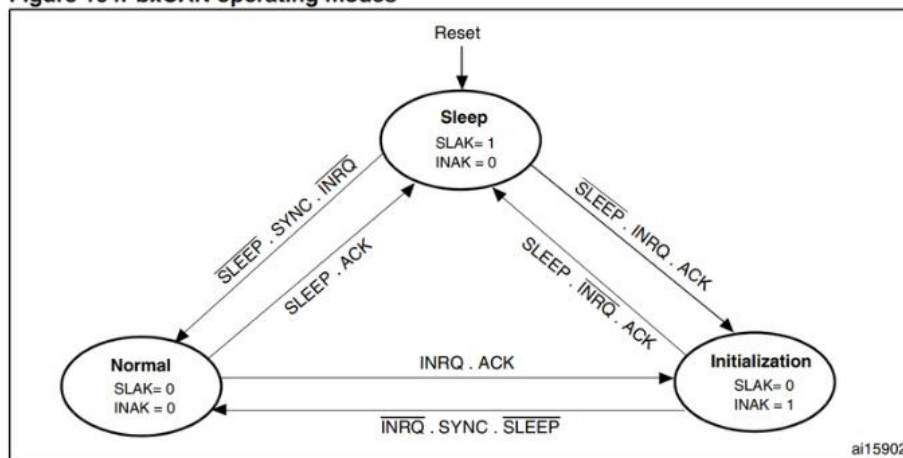
```

1.5. Khởi tạo CAN

1.5.1. Các chế độ hoạt động của CAN:

Bộ điều khiển bxCAN hoạt động dựa trên máy trạng thái (State Machine) với 3 chế độ chính:

Figure 194. bxCAN operating modes



- Chế độ Khởi

tạo (Initialization Mode):

- **Chức năng:** Ngắt toàn bộ giao tiếp trên Bus để cho phép cài đặt các tham số quan trọng (như Baudrate trong CAN_BTR).
- **Trạng thái:** Được xác nhận khi bit INRQ=1 và INAK=1.

- **Chế độ Bình thường (Normal Mode):**

- **Chức năng:** Chế độ vận hành chính thức, cho phép vi điều khiển tham gia truyền và nhận dữ liệu trên mạng.
- **Trạng thái:** Được xác nhận khi bit INRQ=0 và INAK=0 (Đã đồng bộ với Bus).

- **Chế độ Ngủ (Sleep Mode):**

- **Chức năng:** Chế độ mặc định sau khi Reset để tiết kiệm năng lượng (ngắt xung nhịp cấp cho CAN).
- **Trạng thái:** Được xác nhận khi bit SLAK=1 và INAK=0

⇒ **Quy trình khởi tạo chuẩn:** Reset (thoát Sleep) => Initialization (Cấu hình) => Normal (Chạy).

1.5.2. Bật Clock & Cấu hình GPIO:

- **Bật Clock (RCC Setup):** Trước khi thực hiện bất kỳ thao tác ghi nào vào thanh ghi, ta bắt buộc phải cấp xung nhịp cho các khối chức năng liên quan thông qua bộ RCC (Reset and Clock Control):
 - **Cấp Clock cho CAN1:**
 - Ngoài vì CAN nằm trên bus **APB1**.
 - Lệnh cần thực hiện: Bật bit CAN1EN trong thanh ghi RCC_APB1ENR.
 - **Cấp Clock cho GPIO và AFIO:**
 - Các khối này nằm trên bus **APB2**.
 - Cần bật Clock cho GPIOB (vì ta sử dụng chân PB8/PB9)
 - **Quan trọng:** Bắt buộc phải bật Clock cho **AFIO** (Alternate Function I/O). Lý do là ta cần sử dụng chức năng Remap để chuyển chân CAN từ mặc định (PA11/12) sang PB8/PB9.
- **Cấu hình GPIO:** Để kết nối bộ điều khiển CAN bên trong chip với IC thu phát (Transceiver) bên ngoài, ta cần cấu hình đúng chế độ cho các chân TX và RX.

- **Thực hiện Remap chân (Pin Remapping):** Theo mặc định, CAN sử dụng cặp chân PA11/PA12. Tuy nhiên, để tránh xung đột với USB hoặc do thiết kế phần cứng (ví dụ: kit Blue Pill), ta thường chuyển sang cặp chân PB8/PB9.
 - **Thao tác:** Ghi giá trị nhị phân 10 vào các bit CAN_REMAP[1:0] (bit 14:13) của thanh ghi AFIO->MAPR
 - **Lệnh code:** AFIO->MAPR |= (0x2 << 13); (Tương ứng với Remap mode 2).
- **Cấu hình các chân CAN:**
 - **PB8 (CAN_RX):** Mode Input Pull-up
+ Chế độ này giúp giữ mức logic ổn định (Recessive - mức 1) khi đường dây rảnh hoặc bị ngắt kết nối, tránh tín hiệu thả nổi gây nhiễu.
 - **PB9 (CAN_TX):** Mode Alternate Function Push-Pull.
+ Chế độ này cho phép ngoại vi CAN chiếm quyền điều khiển chân PB9 thay vì khối GPIO thông thường, đảm bảo khả năng lái dòng mạnh mẽ để kích hoạt IC thu phát.

- **Chương trình cấu hình Clock và GPIO của dự án:**

```
/**
 * @brief Cấu hình clocks cho CAN1, GPIOB và AFIO
 */
static void can_configure_clocks(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_CAN1EN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN;
}
/**
 * @brief Cấu hình GPIO pins cho CAN (PB8=RX, PB9=TX)
 */
static void can_configure_gpio(void)
{
    /*REMAP (AFIO_MAPR)*/
    AFIO->MAPR &= ~(0x3 << 13); // Clear bit 14, 13
    AFIO->MAPR |= (0x2 << 13); // Set bit 14=1, 13=0
    /* B8 - RX (GPIOB_CRH [3:0])*/
    GPIOB->CRH &= ~(0xF << 0); // Clear 4 bit thấp
    GPIOB->CRH |= (0x8 << 0); // Set cấu hình
    GPIOB->ODR |= (0x1 << 8); // Chọn Pull-up
    /* PB9 - TX (GPIOB_CRH [7:4])*/
    GPIOB->CRH &= ~(0xF << 4); // Clear 4 bit
    GPIOB->CRH |= (0xB << 4); // Set cấu hình
}
```

1.5.3. Vào Mode Init & Thanh ghi MCR:

- Thanh ghi CAN_MCR đóng vai trò là "trung tâm điều hành" của bxCAN, cho phép lập trình viên kiểm soát các hành vi cốt lõi của phần cứng như: quản lý năng lượng, cơ chế tự động phục hồi lỗi và phương thức xử lý hàng đợi.
- Dưới đây là ý nghĩa chi tiết của các bit cấu hình quan trọng:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															DBF
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESET	Reserved							TTCM	ABOM	AWUM	NART	RFLM	TXFP	SLEEP	INRQ
rs								rw	rw	rw	rw	rw	rw	rw	rw

- **Các bit kiểm soát chế độ (Mode Control Bits):**

- **INRQ (Initialization Request - Bit 0):**

- **Chức năng:** Yêu cầu phần cứng chuyển đổi giữa chế độ Khởi tạo và chế độ Hoạt động.
 - **INRQ = 1:** Bắt buộc phải set bit này lên 1 để vào chế độ Khởi tạo trước khi muốn thay đổi bất kỳ tham số nào của CAN (như Baudrate).
 - **INRQ = 0:** Xóa bit này về 0 để báo cho phần cứng biết quá trình cấu hình đã xong, yêu cầu chuyển sang chế độ Normal để bắt đầu chạy.

- **SLEEP (Sleep Request - Bit 1):**

- **Chức năng:** Quản lý chế độ tiết kiệm năng lượng.
 - **SLEEP = 1:** Yêu cầu đưa CAN vào trạng thái ngủ (ngắt xung nhịp).
 - **SLEEP = 0:** Đánh thức CAN dậy để làm việc.

- **AWUM (Automatic Wakeup Mode - Bit 5):**

- **Chức năng:** Cấu hình cơ chế tự động đánh thức khi đang ngủ.
 - **AWUM = 0 (Thủ công):** Hệ thống chỉ thức dậy khi có lệnh từ phần mềm.
 - **AWUM = 1 (Tự động - Khuyến dùng):** Phần cứng sẽ tự động thoát chế độ ngủ ngay khi phát hiện có hoạt động giao tiếp (tín hiệu bus) trên đường dây CAN. Điều này giúp hệ thống phản ứng nhanh mà không cần CPU can thiệp liên tục.

- **Các bit kiểm soát luồng dữ liệu (Data Flow Control Bits):**

- **TXFP (Transmit FIFO Priority - Bit 2):**

- **Chức năng:** Quyết định thứ tự gửi tin nhắn khi có nhiều tin đang chờ trong hàng đợi.
 - **TXFP = 0 (Ưu tiên ID):** Tin nhắn nào có ID nhỏ hơn (độ ưu tiên cao hơn) sẽ được gửi đi trước. Đây là chế độ mặc định và chuẩn của CAN.

- **TXFP = 1 (Ưu tiên thời gian):** Tin nhắn nào được nạp vào hàng đợi trước sẽ được gửi đi trước (First In First Out), bất kể ID là gì.
- **RFLM (Receive FIFO Locked Mode - Bit 3):**
 - **Chức năng:** Quyết định cách xử lý khi bộ đệm nhận bị đầy (Full).
 - **RFLM = 0 (Overwrite - Ghi đè):** Nếu bộ đệm đầy mà có tin mới đến, tin nhắn mới sẽ ghi đè lên tin nhắn cũ nhất chưa được đọc. Dữ liệu cũ bị mất.
 - **RFLM = 1 (Locked - Khóa):** Nếu bộ đệm đầy, tin nhắn mới đến sẽ bị vứt bỏ (Discard). Dữ liệu cũ trong bộ đệm được bảo toàn an toàn cho đến khi CPU đọc xong
- **Các bit quản lý lỗi (Error Management Bits):**
 - **NART (No Automatic Retransmission - Bit 4):**
 - **Chức năng:** Kiểm soát hành vi khi gửi tin thất bại (do nhiễu hoặc va chạm bus).
 - **NART = 0 (Tự động gửi lại - Khuyến dùng):** Nếu gửi lỗi, phần cứng sẽ kiên trì gửi lại tin nhắn đó mãi cho đến khi thành công. Đảm bảo dữ liệu chắc chắn đến nơi.
 - **NART = 1 (Gửi 1 lần):** Chỉ thử gửi đúng 1 lần. Nếu lỗi thì thôi, hủy tin nhắn đó luôn. Dùng trong các ứng dụng thời gian thực khắt khe (ví dụ: tin báo tốc độ động cơ, nếu trễ thì gửi tin mới luôn, không cần tin cũ nữa).
 - **ABOM (Automatic Bus-Off Management - Bit 6):**
 - **Chức năng:** Tự động phục hồi khi mạng bị "sập" (Bus-Off state do quá nhiều lỗi).
 - **ABOM = 0:** Phục hồi thủ công bằng phần mềm.
 - **ABOM = 1 (Tự động):** Phần cứng tự động đếm thời gian và phục hồi kết nối sau khi đường truyền ổn định trở lại (sau chuỗi 128x11 bit Recessive). Giúp hệ thống bền bỉ hơn mà không cần Reset.
- Chương trình hàm Init của dự án:

```
/**
 * @brief Vao che do Initialization
 */
static void can_configure_mcr(void)
{
    CAN1->MCR = (0x1 << 6) | // Bit 6 (ABOM)
                (0x1 << 5) | // Bit 5 (AWUM)
                (0x0 << 4) | // Bit 4 (NART)
                (0x0 << 3) | // Bit 3 (RFLM)
```

```

(0x0 << 2) | // Bit 2 (TXFP)
(0x0 << 1) | // Bit 1 (SLEEP)
(0x1 << 0); // Bit 0 (INRQ)

// Chờ bit INAK (Bit 0 của MSR) lên 1
while (!(CAN1->MSR & (0x1 << 0)));
}

```

1.5.4. Cấu hình ngắt FIFO cho CAN:

- Sử dụng cơ chế ngắt giúp vi điều khiển xử lý dữ liệu ngay lập tức mà không cần kiểm tra liên tục, tiết kiệm tài nguyên CPU. Thanh ghi CAN_IER dùng để kích hoạt các nguồn ngắt này.

- **Các bit ngắt sự kiện nhận (Receive Interrupts):**

- **FMPIEx (FIFO Message Pending):** Ngắt khi có tin nhắn mới trong FIFO. Đây là ngắt quan trọng nhất để đọc dữ liệu.
- **FFIEEx (FIFO Full):** Ngắt cảnh báo khi bộ đệm đã **đầy** (chứa đủ 3 tin).
- **FOVIEEx (FIFO Overrun):** Ngắt báo lỗi **tràn bộ đệm** (tin nhắn mới đến bị mất do không còn chỗ chứa).

- **Kích hoạt NVIC:** Để ngắt thực sự hoạt động, bắt buộc phải cấu hình bộ quản lý ngắt NVIC.

- **Set Priority:** Thiết lập độ ưu tiên cho ngắt CAN.
- **Enable IRQ:** Bật kênh ngắt tương ứng (ví dụ: CAN1_RX1_IRQn cho FIFO 1) để CPU nhảy vào hàm xử lý khi có sự kiện.

- Chương hàm bật ngắt của dự án:

```

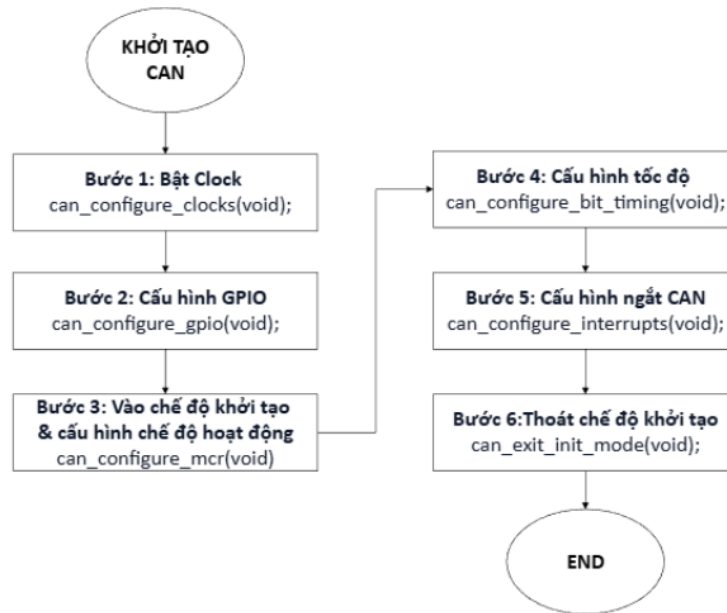
static void can_configure_interrupts(void)
{
    CAN1->IER |= (0x1 << 6) | // FOVIE1: Bật ngắt Tràn
                (0x1 << 5) | // FFIE1: Bật ngắt Đầy
                (0x1 << 4); // FMPIE1: Bật ngắt Có tin mới

    NVIC_SetPriority(CAN1_RX1_IRQn, 2);
    NVIC_EnableIRQ(CAN1_RX1_IRQn);
}

```

1.5.5. Các bước khởi tạo CAN:

- Lưu đồ thuật toán các bước khởi tạo CAN:



- Chương trình khởi tạo CAN của dự án:

```

/**
 * @brief Khởi tạo CAN1 cho Standard CAN 2.0A - 500 kbps @ 72MHz (PB8=RX, PB9=TX)
 */
void can_init(void)
{
    // Buoc 1: Bat clock cho CAN1, GPIOB va AFIO
    can_configure_clocks();

    // Buoc 2: Cau hinh GPIO pins (PB8=RX, PB9=TX) va remap CAN1
    can_configure_gpio();

    // Buoc 3: Vao che do Initialization Mode
    can_configure_mcr();

    // Buoc 4: Cau hinh bit timing cho 500kbps @ 72MHz system clock
    can_configure_bit_timing();

    // Buoc 5: Bat FIFO1 interrupts (message pending, full, overrun)
    can_configure_interrupts();

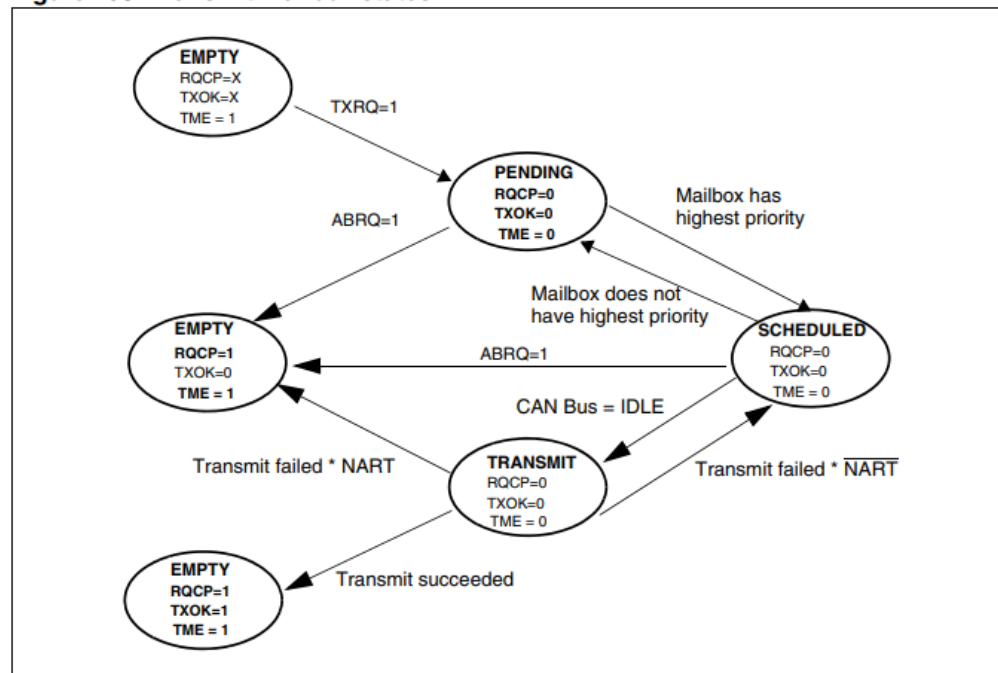
    // Buoc 6: Thoat che do Initialization Mode ve Normal Mode
    can_exit_init_mode();
}
  
```

1.6. Gửi dữ liệu CAN

1.6.1. Luồng thực thi gửi dữ liệu:

- STM32 sử dụng 3 Hộp thư gửi (Tx Mailboxes) hoạt động song song để quản lý hàng đợi tin nhắn. Quá trình này được quản lý tự động bởi phần cứng thông qua máy trạng thái:
 - **Các cờ báo trạng thái quan trọng:**
 - **TME (Empty):** Cờ quan trọng nhất để kiểm tra trước khi gửi.
 - **TME = 1:** Hộp thư trống => Được phép ghi dữ liệu.
 - **TME = 0:** Hộp thư đang bận => Không được ghi dữ liệu.
 - **TXOK (Transmission OK):** Báo hiệu tin nhắn đã gửi thành công (đã nhận được ACK từ mạng).
 - **RQCP (Request Completed):** Báo hiệu quy trình đã kết thúc (dù thành công hay thất bại) để giải phóng hộp thư.
 - **Các trạng thái của Hộp thư (Mailbox States):**
 1. **EMPTY:** Sẵn sàng nhận dữ liệu.
 2. **PENDING:** Đang chờ xếp hàng (Scheduler so sánh độ ưu tiên giữa 3 hộp thư).
 3. **SCHEDULED:** Đã chiến thắng quyền ưu tiên, đang chờ đường truyền rảnh (Bus Idle) để phát.
 4. **TRANSMIT:** Đang đẩy dữ liệu ra đường dây.

Figure 198. Transmit mailbox states



1.6.2. Các thanh ghi phục vụ gửi dữ liệu:

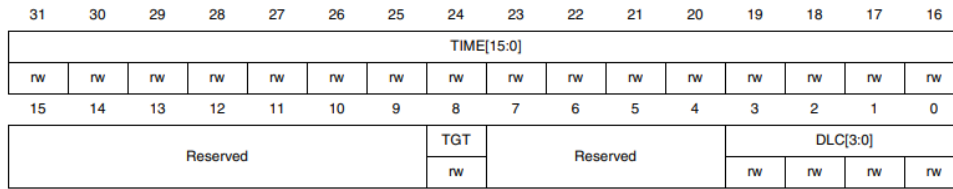
- **Thanh ghi Trạng thái Truyền (CAN_TSR - Transmit Status Register):** Đây là thanh ghi đầu tiên cần được truy cập để kiểm tra tài nguyên phần cứng trước khi thực hiện bất kỳ thao tác ghi nào.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOW2	LOW1	LOW0	TME2	TME1	TME0	CODE[1:0]		ABRQ 2	Reserved			TERR 2	ALST2	TXOK 2	RQCP 2
r	r	r	r	r	r	r	r	rs				rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRQ 1	Reserved Res.			TERR 1	ALST1	TXOK 1	RQCP 1	ABRQ 0	Reserved			TERR 0	ALST0	TXOK 0	RQCP 0
rs				rc_w1	rc_w1	rc_w1	rc_w1	rs				rc_w1	rc_w1	rc_w1	rc_w1

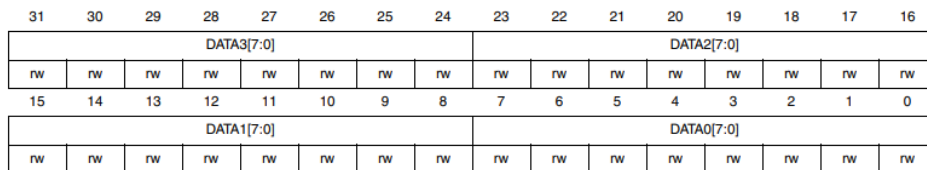
- **Chức năng:** Giám sát trạng thái của cả 3 hộp thư gửi (Mailbox 0, 1, 2).
- **Các bit quan trọng:**
 - **TME (Transmit Mailbox Empty):** Cờ báo trạng thái rỗng. Nếu bit này bằng 1, hộp thư tương ứng đang sẵn sàng nhận dữ liệu. Nếu bằng 0, hộp thư đang bận gửi tin trước đó.
- **Thanh ghi Định danh Hộp thư (CAN_TlR - Transmit Mailbox Identifier Register):** Đây là thanh ghi quan trọng nhất, chứa thông tin định danh của gói tin và bit điều khiển lệnh gửi.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]												IDE	RTR	TXRQ	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

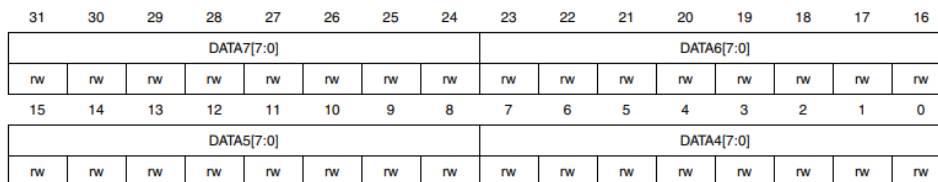
- **Chức năng:** Thiết lập ID và loại khung tin.
- **Các bit quan trọng:**
 - **STID (Standard Identifier):** Đối với Standard CAN, 11-bit ID sẽ được nạp vào các bit cao (từ bit 21 đến bit 31) của thanh ghi này.
 - **RTR (Remote Transmission Request):** Bit xác định loại khung tin là Data Frame (chứa dữ liệu) hay Remote Frame (yêu cầu dữ liệu).
 - **TXRQ (Transmit Request):** Ngay khi phần mềm set bit này lên 1, phần cứng sẽ khóa hộp thư và bắt đầu quy trình đẩy dữ liệu ra đường truyền.
- **Thanh ghi Độ dài Dữ liệu (CAN_TDTxR - Transmit Data Length Control)**



- **Chức năng:** Quy định kích thước của gói tin sắp gửi.
- **DLC (Data Length Code):** Chứa giá trị từ 0 đến 8, tương ứng với số byte dữ liệu (Payload) sẽ được gửi đi. Bắt buộc phải cấu hình đúng số lượng này để bên nhận giải mã chính xác.
- **Các thanh ghi Dữ liệu (CAN_TDLxR và CAN_TDHxR):**
Do mỗi gói tin CAN chứa tối đa 8 byte dữ liệu, phần cứng chia không gian lưu trữ thành 2 thanh ghi 32-bit riêng biệt:



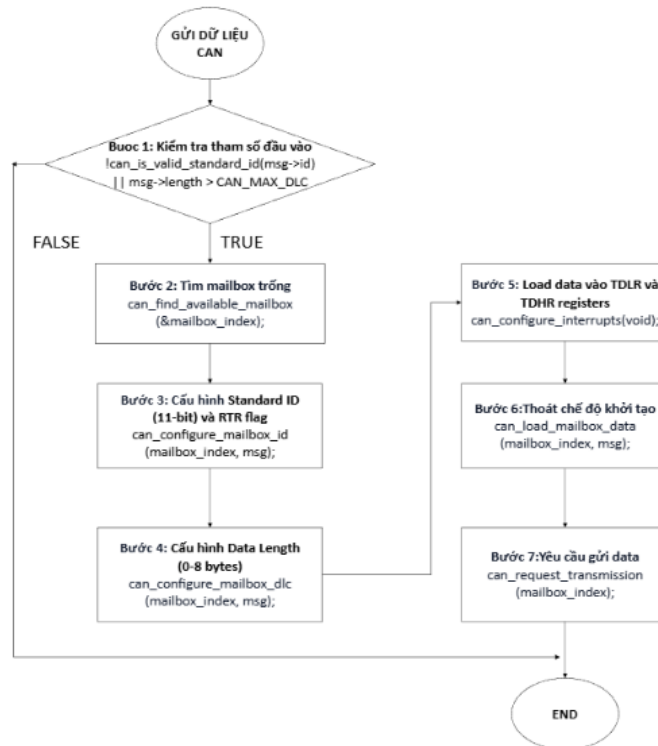
- **CAN_TDLxR (Transmit Data Low Register):** Chứa 4 byte dữ liệu thấp (Data byte 0 đến Data byte 3).DLC (Data Length Code).



- **CAN_TDHxR (Transmit Data High Register):** Chứa 4 byte dữ liệu cao (Data byte 4 đến Data byte 7).

1.6.3. Các bước gửi dữ liệu:

- Lưu đồ thuật toán các bước gửi dữ liệu:



- Chương trình các bước gửi dữ liệu:

```

/**
 * @brief Gui Standard CAN message
 */
bool can_transmit(can_message_t *msg)
{
    // Bước 1: Kiểm tra tham số đầu vào hợp lệ
    if (!can_is_valid_standard_id(msg->id) || msg->length > CAN_MAX_DLC)
    {
        return false; // ID > 0x7FF hoặc DLC > 8 - không hợp lệ
    }

    uint32_t mailbox_index;

    // Bước 2: Tìm mailbox trống (0, 1 hoặc 2)
    if (!can_find_available_mailbox(&mailbox_index))
    {
        return false; // Tất cả 3 mailbox đang bận - thu lại sau
    }

    // Bước 3: Cấu hình Standard ID (11-bit) và RTR flag
    can_configure_mailbox_id(mailbox_index, msg);

    // Bước 4: Cấu hình Data Length Code (0-8 bytes)

```

```

    can_configure_mailbox_dlc(mailbox_index, msg);

    // Buoc 5: Load data vao TDLR va TDHR registers (neu khong phai RTR)
    can_load_mailbox_data(mailbox_index, msg);

    // Buoc 6: Request transmission - set TXRQ bit
    can_request_transmission(mailbox_index);

    return true;
}

/**
 * @brief Kiem tra Standard ID hop le
 */
bool can_is_valid_standard_id(uint16_t id)
{
    return (id <= CAN_STANDARD_ID_MASK);
}

static bool can_find_available_mailbox(uint32_t *mailbox_index)
{
    uint32_t tsr = CAN1->TSR;

    // Kiem tra xem co hom thu nao ranh khong (bit TME = 1 la ranh)
    if (!((tsr & CAN_TSR_TME0) || (tsr & CAN_TSR_TME1) || (tsr & CAN_TSR_TME2)))
    {
        return false; // Ca 3 hom thu deu ban -> Bao loi
    }

    // Phan cung tu dong luu index cua hom thu ranh vao bit 24-25
    *mailbox_index = (tsr >> 24) & 0x03;
    return true;
}

/**
 * @brief Cai dat ID
 */
static void can_configure_mailbox_id(uint32_t mailbox_index, const can_message_t *msg)
{
    // ID chuan nam o 11 bit cao nhat -> Dich trai 21 bit CAN_TlXR
    CAN1->sTxMailBox[mailbox_index].TlR =
        ((uint32_t)(msg->id & CAN_STANDARD_ID_MASK) << 21) |
        (msg->rtr ? CAN_Tl0R_RTR : 0);
}

```

```

/**
 * @brief Cài đặt độ dài dữ liệu (DLC)
 */
static void can_configure_mailbox_dlc(uint32_t mailbox_index, const can_message_t *msg)
{
    // Chỉ lấy 4 bit thấp nhất làm độ dài CAN_TDTxR
    CAN1->sTxMailBox[mailbox_index].TDTR = (msg->length & 0x0F);
}

/**
 * @brief Nạp dữ liệu vào hộp thư
 */
static void can_load_mailbox_data(uint32_t mailbox_index, const can_message_t *msg)
{
    if (!msg->rtr && msg->length > 0)
    {
        // Xóa sạch dữ liệu cũ trong 2 thanh ghi Low và High
        CAN1->sTxMailBox[mailbox_index].TDLR = 0;
        CAN1->sTxMailBox[mailbox_index].TDHR = 0;

        // Chia dữ liệu vào 2 thanh ghi (TDLR chứa byte 0-3, TDHR chứa byte 4-7)
        for (uint8_t i = 0; i < msg->length && i < CAN_STANDARD_DATA_LENGTH; i++)
        {
            if (i < 4) // 4 byte đầu vào thanh ghi Thấp
            {
                CAN1->sTxMailBox[mailbox_index].TDLR |= ((uint32_t)msg->data[i] << (i * 8));
            }
            else // 4 byte sau vào thanh ghi Cao
            {
                CAN1->sTxMailBox[mailbox_index].TDHR |= ((uint32_t)msg->data[i] << ((i - 4) * 8));
            }
        }
    }
}

/**
 * @brief Ra lệnh gửi đi
 */
static void can_request_transmission(uint32_t mailbox_index)
{
    // Bật bit TXRQ lên 1 -> Phản ứng bắt đầu gửi ngay lập tức
    CAN1->sTxMailBox[mailbox_index].TIR |= CAN_TI0R_TXRQ;
}

```

1.7. Nhận dữ liệu CAN

1.7.1. Luồng thực thi nhận dữ liệu:

- STM32 sử dụng cơ chế **FIFO 3 cấp** (First In First Out) để tạo vùng đệm nhận, giúp tránh mất mát dữ liệu khi CPU chưa kịp xử lý.
- **Các cờ trạng thái quan trọng (Status Flags):**
 - **FMP (FIFO Message Pending):**
 - Biến đếm số lượng tin nhắn đang chờ trong hàng đợi.
 - Giá trị từ 0 (Rỗng) đến 3 (Đầy).
 - **FOVR (FIFO Overrun):** Cờ báo lỗi Tràn bộ đệm.
 - Kích hoạt (FOVR=1) khi bộ đệm đã đầy (FMP=3) mà vẫn có thêm tin nhắn mới đến => Tin mới bị mất.
- **Quy trình xử lý (Process Flow):**
 Quy trình tuân thủ vòng đời: **Nhận => Đọc => Giải phóng.**
 1. **Nhận (Hardware):** Khi có tin nhắn hợp lệ, phần cứng tự động nạp vào FIFO và tăng biến đếm FMP.
 2. **Giải phóng (Software):** Đây là bước quan trọng nhất. Sau khi CPU đọc xong dữ liệu, bắt buộc phải set bit **RFOM = 1** (Release FIFO Output Mailbox).

1.7.2. Các thanh ghi phục vụ gửi dữ liệu:

- Trong quá trình nhận, phần cứng tự động nạp dữ liệu từ đường truyền vào các thanh ghi của **FIFO 0 hoặc 1**. Nhiệm vụ của phần mềm là đọc dữ liệu từ các thanh ghi này ra biến và giải phóng bộ đệm.
- **Thanh ghi Trạng thái FIFO x (CAN_RFxR - Receive FIFO x Register):** Đây là thanh ghi quản lý trạng thái và điều khiển luồng của bộ đệm FIFO 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RFOM1	FOVR1	FULL1	Res.		FMP1[1:0]
										rs	rc_w1	rc_w1			r

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RFOM0	FOVR0	FULL0	Res.		FMP0[1:0]
										rs	rc_w1	rc_w1			r

- **FMPx (FIFO Message Pending - Bits 1:0):** Biến đếm số lượng tin nhắn đang chờ.
 - **Chức năng:** Kiểm tra xem có tin nhắn nào trong hộp thư không. Nếu $FMPx > 0$, hàm ngắt sẽ tiến hành đọc.
- **RFOMx (Release FIFO Output Mailbox - Bit 5):** Bit lệnh giải phóng bộ đệm.

- **Chức năng:** Sau khi đọc xong, bắt buộc phải set bit này lên 1. Phần cứng sẽ xóa tin nhắn vừa đọc và giải phóng chỗ trống cho tin nhắn mới.
- **Thanh ghi Định danh FIFO x (CAN_RIxR - Receive FIFO Mailbox Identifier Register):** Thanh ghi này chứa thông tin định danh (ID) và loại khung tin của tin nhắn vừa nhận được.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]												IDE		RTR	Res.
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	

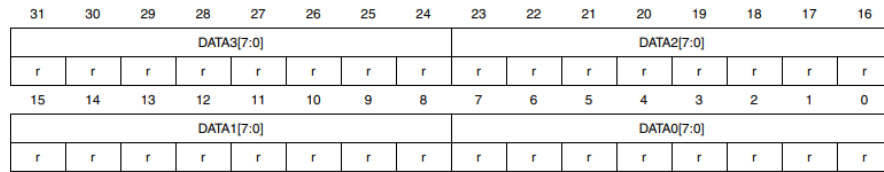
- **IDE (Identifier Extension - Bit 2):** Cờ phân loại ID.
 - **IDE = 0:** Standard Frame (11-bit).
 - **IDE = 1:** Extended Frame (29-bit).
 - **Các lấy ID:**
 - **Với Standard ID (11-bit):** ID nằm ở 11 bit cao nhất (STID).
 - **Với Extended ID (29-bit):** ID nằm ở cả phần STID và EXID, bắt đầu từ bit 3.
 - **RTR (Remote Transmission Request - Bit 1):**
 - Dùng để kiểm tra xem gói tin nhận được là dữ liệu thực (Data Frame) hay chỉ là yêu cầu (Remote Frame).
- **Thanh ghi Độ dài & Chỉ số bộ lọc (CAN_RDTxR)**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIME[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMI[7:0]								Reserved				DLC[3:0]			
r	r	r	r	r	r	r	r					r	r	r	r

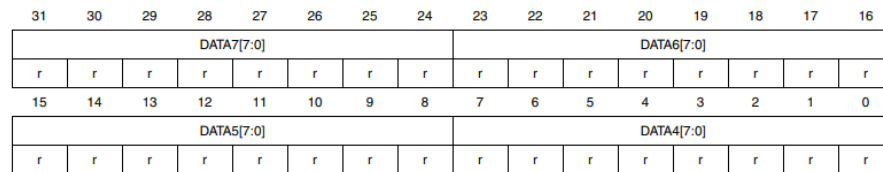
- **DLC (Data Length Code - Bits 3:0):** Độ dài dữ liệu
 - **FMI (Filter Match Index - Bits 15:8):** Chỉ số bộ lọc khớp.
 - Khi dùng Extended ID hoặc Mask mode, nhiều ID khác nhau có thể lọt qua cùng một FIFO.
- **Các thanh ghi Dữ liệu (CAN_RDLxR & CAN_RDHxR)**

Hai thanh ghi này chứa tối đa 8 byte dữ liệu (Payload), không phụ thuộc vào việc ID là 11-bit hay 29-bit.

 - **CAN_RDLxR (Low Register):** Chứa Data Byte 0 đến Data Byte 3.

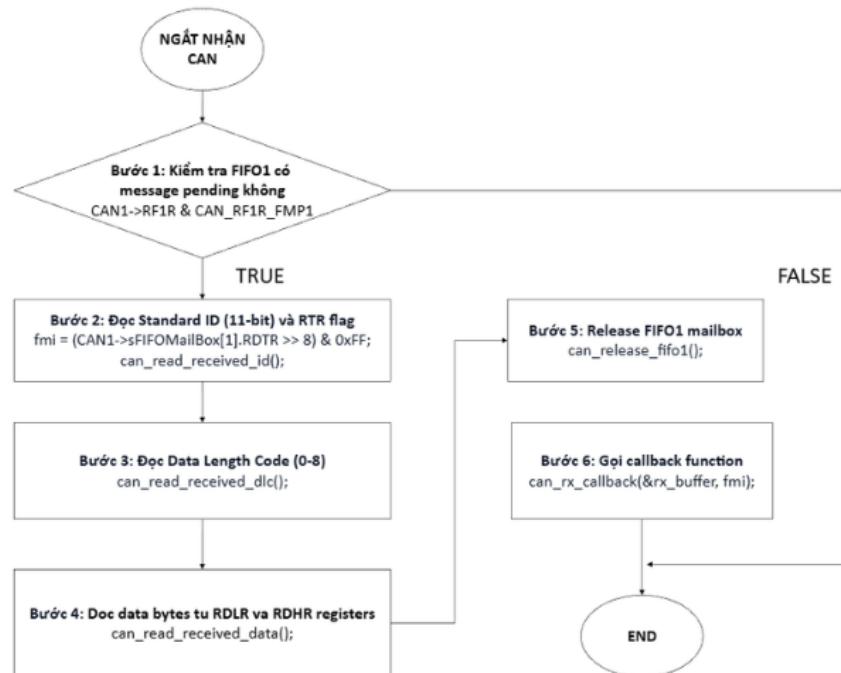


- **CAN_RDHxR (High Register):** Chứa Data Byte 4 đến Data Byte 7.



1.7.3. Các bước nhận dữ liệu:

- Lưu đồ thuật toán các bước nhận dữ liệu của dự án:



- Chương trình các bước nhận dữ liệu của dự án:

```

/**
 * @brief Interrupt handler cho Standard CAN RX FIFO1
 * @note Hàm này được gọi từ hardware interrupt khi có message mới
 */
void CAN1_RX1_IRQHandler(void)

```

```

{
    // Buoc 1: Kiem tra FIFO1 co message pending khong
    if (CAN1->RF1R & CAN_RF1R_FMP1)
    {
        // Buoc 2: Doc Standard ID (11-bit) va RTR flag tu RIR register
        fmi = (CAN1->sFIFOMailBox[1].RDTR >> 8) & 0xFF; // FMI
        can_read_received_id();
        // Buoc 3: Doc Data Length Code (0-8) tu RDTR register
        can_read_received_dlc();
        // Buoc 4: Doc data bytes tu RDLR va RDHR registers
        can_read_received_data();
        // Buoc 5: Release FIFO1 mailbox de nhan message tiep theo
        can_release_fifo1();

        // Buoc 6: Goi callback function de xu ly message
        can_rx_callback(&rx_buffer, fmi);
    }
}
**
* @brief Doc ID tu FIFO1
*/
static void can_read_received_id(void)
{
    rx_buffer.id = (uint16_t)((CAN1->sFIFOMailBox[1].RIR >> 21) &
CAN_STANDARD_ID_MASK);
    rx_buffer.rtr = (CAN1->sFIFOMailBox[1].RIR & CAN_R10R_RTR) ? true : false;
}
/**
* @brief Doc DLC tu FIFO1
*/
static void can_read_received_dlc(void)
{
    rx_buffer.length = CAN1->sFIFOMailBox[1].RDTR & 0x0F;

    if (rx_buffer.length > CAN_MAX_DLC)
    {
        rx_buffer.length = CAN_MAX_DLC;
    }
}
/**
* @brief Doc data tu FIFO1
*/
static void can_read_received_data(void)
{
    // Clear buffer truoc

```

```

memset(rx_buffer.data, 0, CAN_STANDARD_DATA_LENGTH);
// Chi doc data neu khong phai RTR frame
if (!rx_buffer.rtr && rx_buffer.length > 0)
{
    uint32_t rdlr = CAN1->sFIFOMailBox[1].RDLR;
    uint32_t rdhr = CAN1->sFIFOMailBox[1].RDHR;
    for (uint8_t i = 0; i < rx_buffer.length; i++)
    {
        if (i < 4)
        {
            rx_buffer.data[i] = (rdlr >> (i * 8)) & 0xFF;
        }
        else
        {
            rx_buffer.data[i] = (rdhr >> ((i - 4) * 8)) & 0xFF;
        }
    }
}

```